

Refactorización de la clase `Node`
Extracción de responsabilidades mediante el patrón *Extract Class*
`org.openmarkov.core` — paquete `model.network`

Manuel Arias Calleja

Marzo de 2026

Índice

1. Introducción	2
2. Fundamentos teóricos	2
2.1. Principio de Responsabilidad Única (SRP)	2
2.2. Patrón <i>Extract Class</i>	2
2.3. Clases de utilidad estáticas vs. composición	2
3. Situación anterior: Node con 1.037 líneas	3
3.1. Métodos problemáticos	5
4. Refactorización realizada	5
4.1. Visión general	5
4.2. Fase 1: Extracción de <code>NodeAbsorptionHandler</code>	5
4.3. Fase 2: Extracción de <code>VariableTypeConverter</code>	6
4.4. Fase 3: Extracción de <code>UtilityFunctionComputer</code>	6
4.5. Fase 4: Migración de <code>setPotentialConsistently()</code>	7
4.6. Fase 5: Eliminación de código muerto	7
5. Situación después: Node con 670 líneas	7
5.1. Responsabilidades que permanecen en <code>Node</code>	8
6. Ventajas del nuevo diseño	8
6.1. Cumplimiento del Principio de Responsabilidad Única	8
6.2. Reducción de la complejidad	8
6.3. Eliminación de código duplicado	9
6.4. Testabilidad	9
6.5. Menor riesgo de efectos secundarios	9
7. Problemas que se evitan	9
7.1. Problema de la clase dios (<i>God Class</i>)	9
7.2. Problema del acoplamiento indirecto	9
7.3. Problema de las dependencias excesivas	9
8. Verificación	10
8.1. Tests automáticos	10
8.2. Corrección adicional: <code>PotentialUtils.getPotentialName()</code>	10
9. Resumen de ficheros afectados	11

1. Introducción

Este documento describe la refactorización de la clase `Node` del módulo `org.openmarkov.core`, que pasó de 1.037 líneas a 670 líneas mediante la extracción de lógica compleja a tres nuevas clases de utilidad.

La refactorización sigue el patrón *Extract Class* [2] y se fundamenta en el *Principio de Responsabilidad Única* (SRP) [1]. Ambos conceptos se explican en la sección 2 antes de describir los cambios concretos.

2. Fundamentos teóricos

2.1. Principio de Responsabilidad Única (SRP)

El *Single Responsibility Principle* establece que **una clase debe tener una única razón para cambiar** [1]. Si una clase acumula múltiples responsabilidades, un cambio en una de ellas puede introducir errores en las demás, porque comparten el mismo código fuente, los mismos campos y los mismos métodos auxiliares.

Ejemplo intuitivo: imaginemos un cuchillo suizo que además de cortar, destornilla, abre botellas y tiene lupa. Si necesitamos mejorar la hoja del cuchillo, corremos el riesgo de romper el destornillador. En software, cada “herramienta” debería ser un objeto independiente.

Cuando una clase viola SRP, los síntomas típicos son:

- La clase tiene muchas líneas (cientos o miles).
- Los métodos se agrupan en bloques temáticos que no se relacionan entre sí.
- Los cambios en una parte de la clase obligan a re-testear partes no relacionadas.
- Es difícil nombrar la clase con un sustantivo único que describa todo lo que hace.

2.2. Patrón *Extract Class*

El patrón *Extract Class* [2] es la técnica de refactorización que se aplica cuando una clase tiene demasiadas responsabilidades. Consiste en:

1. Identificar un grupo cohesivo de campos y métodos que forman una responsabilidad independiente.
2. Crear una nueva clase con esos campos y métodos.
3. En la clase original, sustituir la lógica extraída por una delegación a la nueva clase (o eliminarla si los llamadores pueden usar la nueva clase directamente).

En nuestro caso, los métodos extraídos no usan campos privados de `Node` —solo acceden a métodos públicos como `getPotentials()`, `getParents()`, `getProbNet()`—, lo que permite moverlos a **clases de utilidad estáticas** que reciben el `Node` como parámetro. Esta variante es más sencilla que la composición (donde `Node` contendría una referencia a la nueva clase) porque no añade campos ni modifica la construcción de `Node`.

2.3. Clases de utilidad estáticas vs. composición

Existen dos formas principales de aplicar *Extract Class*:

	Composición	Utilidad estática
Estructura	<code>Node</code> contiene un campo <code>NodeAbsorptionHandler handler</code>	<code>NodeAbsorptionHandler</code> es una clase con métodos estáticos
Llamada	<code>handler.absorb(variable)</code>	<code>NodeAbsorptionHandler.absorb(node, variable)</code>
Ventaja	Más orientada a objetos; permite polimorfismo (diferentes handlers)	Sin estado; no modifica los constructores de <code>Node</code> ; más simple
Desventaja	Añade un campo a <code>Node</code> ; aumenta el coste de construcción	Menos flexible; los métodos no pueden sobreescribirse

Para esta refactorización elegimos la variante estática porque:

- Los métodos extraídos no necesitan estado propio.
- No se prevé la necesidad de múltiples implementaciones (polimorfismo).
- No se modifican los constructores de `Node`, que es una clase usada en más de 130 ficheros.
- Es coherente con el estilo del proyecto, que ya usa clases de utilidad estáticas como `ProbNetOperations` y `DiscretePotentialOperations`.

3. Situación anterior: Node con 1.037 líneas

La clase `Node` acumulaba al menos seis responsabilidades distintas en un solo fichero:

1. **Identidad del nodo:** `variable`, tipo de nodo, `red`, `hash`, `equals`, `toString`.
2. **Gestión de potenciales (CRUD):** añadir, eliminar, consultar potenciales.
3. **Delegación al grafo:** métodos de conveniencia como `getParents()`, `getChildren()`, `isParent()`.
4. **Metadatos:** propósito, relevancia, comentario, política, propiedades adicionales.
5. **Presentación visual:** coordenadas X e Y.
6. **Lógica compleja de transformación:** absorción de nodos, conversión de tipos de variable, cálculo de funciones de utilidad, aplicación de restricciones de enlace.

Las responsabilidades 1–5 son legítimas para una clase que representa un nodo en una red probabilística. La responsabilidad 6, sin embargo, contiene **308 líneas de lógica de dominio compleja** que no tiene relación con la identidad ni con el CRUD del nodo.

La figura 1 muestra el diagrama UML de `Node` antes de la refactorización.

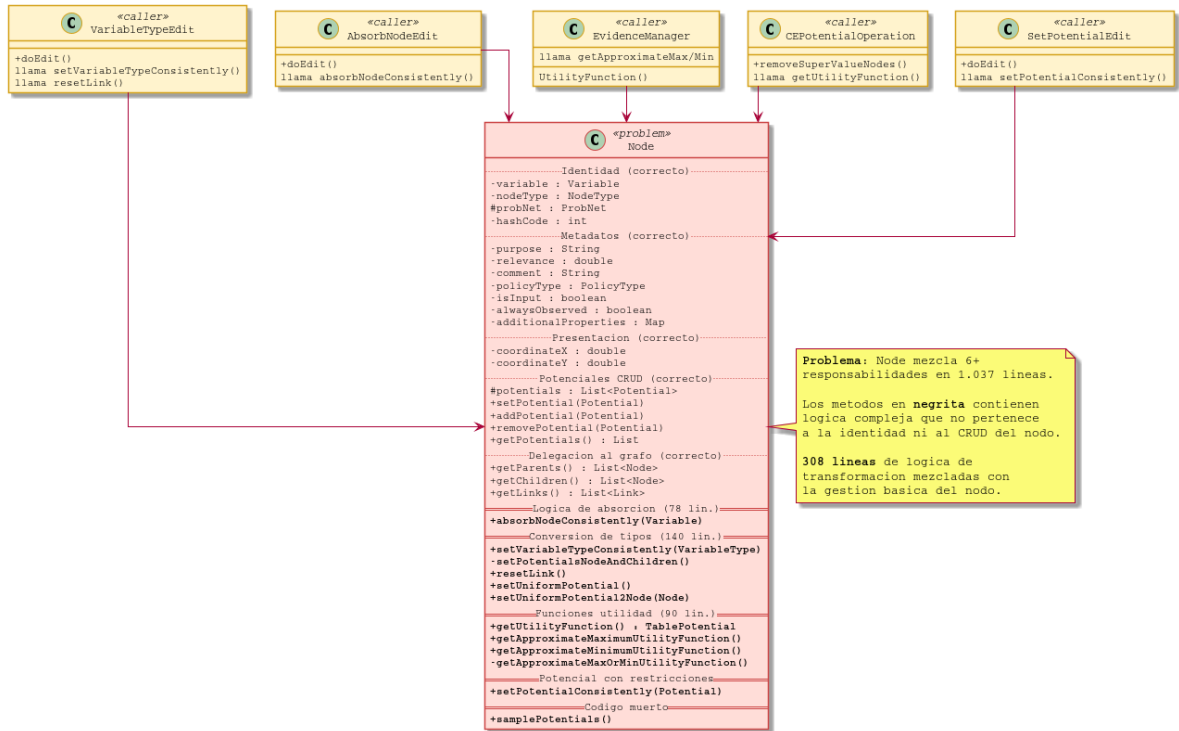


Figura 1: Node antes de la refactorización. La clase (en rojo) contiene métodos de lógica compleja que no pertenecen a la identidad del nodo. Los llamadores (en amarillo) dependen directamente de Node para toda la funcionalidad.

3.1. Métodos problemáticos

Método	Líneas	Problema
absorbNodeConsistently()	78	Multiplica/maximiza potenciales de un nodo padre sobre su hijo utilidad; añade arcos al grafo. Lógica de inferencia, no de identidad de nodo.
setVariableTypeConsistently()	60	Cambia el tipo de variable y actualiza en cascada los potenciales del nodo y de todos sus hijos.
setPotentialsNodeAndChildren()	45	Método auxiliar privado de la conversión de tipos.
resetLink()	17	Limpia condiciones reveladas y restricciones de los arcos. Acoplado con la conversión de tipos.
setUniformPotential()	30	Crea una tabla de probabilidades uniforme manualmente en lugar de usar <code>PotentialOperations.getUniformPotential()</code> que ya existe en el proyecto.
getUtilityFunction()	28	Calcula recursivamente la función de utilidad para nodos supervalor. Lógica de inferencia.
getApproximate*UtilityFunction()	50	Aproxima el máximo/mínimo de la función de utilidad. Lógica de inferencia.
setPotentialConsistently()	12	Asigna un potencial y aplica restricciones de enlace. Lógica de restricciones.
samplePotentials()	6	Código muerto: sin ningún llamador externo.

Tabla 1: Métodos de lógica compleja en `Node` antes de la refactorización. Total: ≈ 326 líneas.

4. Refactorización realizada

4.1. Visión general

La figura 2 ilustra conceptualmente la separación de responsabilidades aplicada.

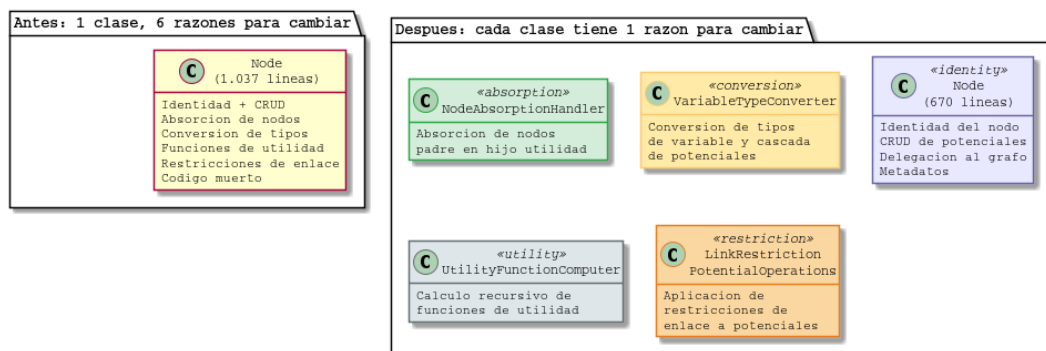


Figura 2: Antes: una clase con 6 razones para cambiar. Después: cada clase tiene una única razón para cambiar (SRP).

4.2. Fase 1: Extracción de `NodeAbsorptionHandler`

Responsabilidad extraída: absorción de un nodo padre (chance o decisión) en su hijo utilidad.

Antes:

Listing 1: Llamada en `AbsorbNodeEdit` antes de la refactorización

```
1 // AbsorbNodeEdit.doEdit()  
2 absorbedNode.absorbNodeConsistently(absorbedVariable);
```

Después:

Listing 2: Llamada después de la refactorización

```
1 // AbsorbNodeEdit.doEdit()  
2 NodeAbsorptionHandler.absorbNodeConsistently(absorbedNode, absorbedVariable);
```

El método `absorbNodeConsistently()` contenía lógica de multiplicación y marginalización de potenciales, creación de `ExactDistrPotential`, y manipulación de arcos del grafo. Ninguna de estas operaciones accede a campos privados de `Node`: todas usan métodos públicos como `getPotentials()`, `getParents()` y `getProbNet().addLink()`.

Único llamador: `AbsorbNodeEdit.doEdit()` — actualizado directamente.

4.3. Fase 2: Extracción de `VariableTypeConverter`

Responsabilidad extraída: conversión del tipo de variable de un nodo y actualización en cascada de sus potenciales y los de sus hijos.

Se extrajeron cinco métodos:

- `setVariableTypeConsistently()` → `VariableTypeConverter.convertVariableType()`
- `resetLink()` → `VariableTypeConverter.resetLinks()`
- `setPotentialsNodeAndChildren()` (privado, se mueve junto)
- `setUniformPotential2Node()` (se integra como método privado)
- `setUniformPotential()` — **eliminado**. Creaba una tabla uniforme manualmente (30 líneas) duplicando la funcionalidad de `PotentialOperations.getUniformPotential()` que ya existía en el proyecto. `VariableTypeConverter` usa el método existente.

Único llamador: `VariableTypeEdit.doEdit()` — actualizado directamente.

4.4. Fase 3: Extracción de `UtilityFunctionComputer`

Responsabilidad extraída: cálculo recursivo de funciones de utilidad para nodos supervalor (nodos con dos o más padres utilidad combinados mediante `SumPotential` o `ProductPotential`).

Se extrajeron cuatro métodos:

- `getUtilityFunction()` → `UtilityFunctionComputer.computeUtilityFunction()`
- `getApproximateMaximumUtilityFunction()` → `UtilityFunctionComputer.approximateMaxUtility()`
- `getApproximateMinimumUtilityFunction()` → `UtilityFunctionComputer.approximateMinUtility()`
- `getApproximateMaxOrMinUtilityFunction()` (privado, se mueve junto)

Qué se queda en `Node`: el método `isSuperValueNode()` permanece porque es una **consulta estructural** del nodo (“¿tiene dos o más padres utilidad?”), análoga a `getParents()` o `getChildren()`. Además, tiene 6+ llamadores en módulos distintos (`ProductPotential`, `SumPotential`, `TreeADDPotential`, `NoSuperValueNode`, `CEPotentialOperation`, `HybridElimination`).

Llamadores migrados:

- `CEPotentialOperation.removeSuperValueNodes()` (módulo `inference`).
- `EvidenceManager` (módulo `gui`).

4.5. Fase 4: Migración de `setPotentialConsistently()`

El método `setPotentialConsistently()` (12 líneas) asignaba un potencial al nodo y después aplicaba las restricciones de enlace si el potencial era una `TablePotential` y el nodo no era de decisión.

Se movió a `LinkRestrictionPotentialOperations` (clase estática ya existente que gestiona todas las operaciones de restricción de enlace) como nuevo método estático `setPotentialWithRestrictions(Node Potential)`.

Llamadores migrados:

- `SetPotentialEdit.doEdit()`, `undo()`, `redo()` (módulo `core`).
- `PotentialEditDialog.setPotentialInNode()` y `removePotentialOnClose()` (módulo `gui`).

4.6. Fase 5: Eliminación de código muerto

- `samplePotentials()` (6 líneas): sin ningún llamador externo confirmado — eliminado.
- Código comentado en `toString()`: bloques de `case` `COST`, `EFFECTIVENESS`, `CE` que llevaban comentados desde hace años — eliminados.
- Todos los delegados deprecados que se introdujeron temporalmente durante las fases 1–4 fueron eliminados tras verificar que ningún llamador los usaba.

5. Situación después: Node con 670 líneas

La figura 3 muestra la arquitectura resultante.

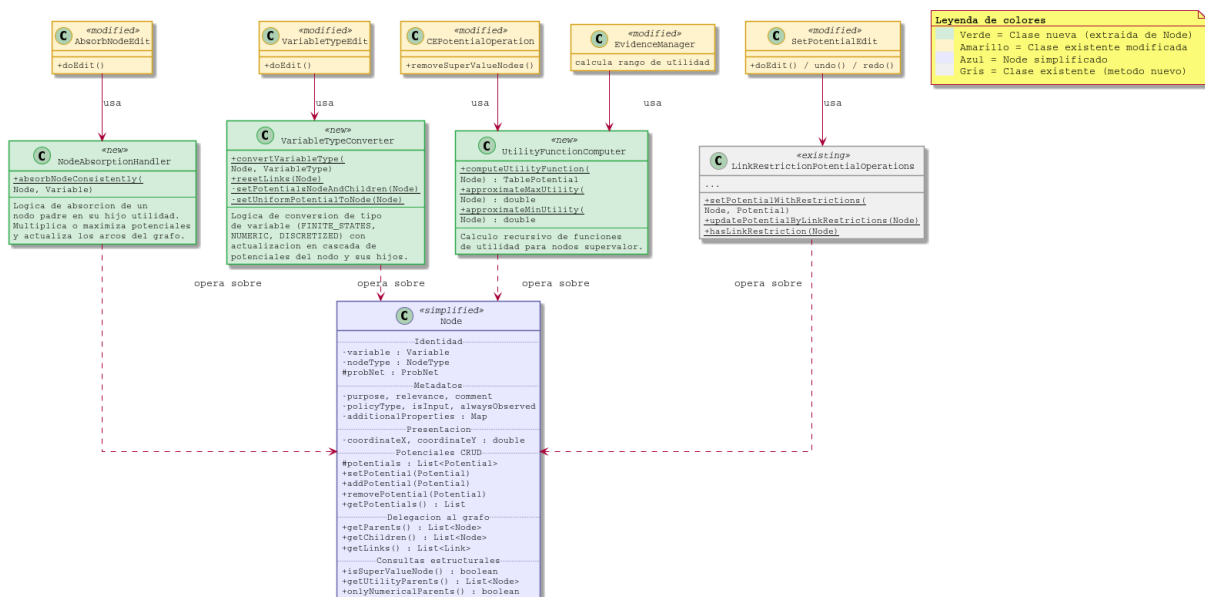


Figura 3: `Node` después de la refactorización. Las clases nuevas (en verde) encapsulan la lógica extraída. Los llamadores (en amarillo) usan directamente las nuevas clases. `Node` (en azul) conserva solo sus responsabilidades legítimas.

5.1. Responsabilidades que permanecen en Node

Responsabilidad	Líneas	Justificación
Campos + constructores + clone	90	Estado esencial del nodo.
Identidad (get/set variable, tipo, equals, hash, toString)	100	Contrato básico de la clase.
CRUD de potenciales	60	Operaciones simples sobre la lista de potenciales.
Delegación al grafo	80	Métodos de conveniencia de una línea: <code>node.getParents()</code> es más legible que <code>node.getProbNet().getParents(node)</code> .
Consultas estructurales	40	<code>isSuperValueNode()</code> , <code>getUtilityParents()</code> , <code>onlyNumericalParents()</code> : propiedades del nodo, no lógica de transformación.
Metadatos y coordenadas	60	Getters/setters de propiedades simples. Las coordenadas se mantienen en <code>Node</code> porque son leídas/escritas por los módulos <code>io</code> (serialización), <code>gui</code> (renderizado), <code>core</code> (copia) y <code>learning</code> .

Tabla 2: Responsabilidades que permanecen en `Node` tras la refactorización.

6. Ventajas del nuevo diseño

6.1. Cumplimiento del Principio de Responsabilidad Única

Cada clase tiene ahora una única razón para cambiar:

Clase	Razón para cambiar
<code>Node</code>	Cómo se identifica y gestiona un nodo en la red
<code>NodeAbsorptionHandler</code>	Cómo se absorbe un nodo padre en su hijo utilidad
<code>VariableTypeConverter</code>	Cómo se convierte el tipo de una variable y se actualizan los potenciales en cascada
<code>UtilityFunctionComputer</code>	Cómo se calcula la función de utilidad de un nodo supervisor

6.2. Reducción de la complejidad

Métrica	Antes	Después
Líneas en <code>Node.java</code>	1.037	670
Métodos públicos en <code>Node</code>	≈50	≈35
Imports en <code>Node.java</code>	15	9
Dependencias de <code>Node</code> hacia paquete <code>potential</code>	6 clases	1 clase

La reducción de imports es un indicador indirecto de que `Node` depende de menos conceptos. Antes importaba `TablePotential`, `ExactDistrPotential`, `SumPotential`, `UniformPotential`, `PotentialRole` y `DiscretePotentialOperations`. Ahora solo importa `Potential` (la interfaz base).

6.3. Eliminación de código duplicado

El método `setUniformPotential()` (30 líneas) duplicaba la funcionalidad de `PotentialOperations.getUniformPotential()` que ya existía en el proyecto. La nueva clase `VariableTypeConverter` usa el método existente, eliminando la duplicación.

6.4. Testabilidad

Las clases extraídas son clases de utilidad estáticas sin estado. Esto facilita enormemente su test unitario: basta crear un `Node` de prueba con los potenciales adecuados y llamar al método estático. No es necesario simular la construcción completa de un `Node` dentro de una `ProbNet` para testear la lógica de absorción, conversión o cálculo de utilidad.

6.5. Menor riesgo de efectos secundarios

Antes de la refactorización, cualquier cambio en `Node` —por ejemplo, añadir un nuevo tipo de potencial o cambiar el formato de `toString()`— requería revisar 1.037 líneas para asegurar que no se rompía la lógica de absorción o de conversión de tipos. Ahora, esa lógica está aislada en sus propias clases y no se ve afectada por cambios en la identidad o presentación del nodo.

7. Problemas que se evitan

7.1. Problema de la clase dios (*God Class*)

Una *clase dios* es una clase que “lo sabe todo y lo hace todo”. Es un antipatrón reconocido [3] porque:

- Concentra la lógica en un solo punto, creando un cuello de botella para el desarrollo: varios desarrolladores necesitan modificar el mismo fichero simultáneamente.
- Dificulta la comprensión: un desarrollador nuevo necesita entender 1.000+ líneas para modificar cualquier aspecto del nodo.
- Impide la reutilización: la lógica de absorción estaba atada a la clase `Node` y no podía usarse en otro contexto sin depender de toda la clase.

Con la refactorización, `Node` deja de ser una clase dios. Su tamaño pasa de 1.037 a 670 líneas, y la lógica compleja se distribuye en clases especializadas.

7.2. Problema del acoplamiento indirecto

Antes, clases como `AbsorbNodeEdit`, `VariableTypeEdit` y `SetPotentialEdit` dependían de `Node` para funcionalidad de inferencia (multiplicación de potenciales), conversión de tipos y restricciones de enlace. Esto significaba que un cambio en cómo `Node` gestiona sus potenciales podía romper la lógica de absorción, aunque ambas cosas no tuvieran relación.

Ahora, cada clase de edición depende de la clase de utilidad específica que necesita. Si cambia la lógica de absorción, solo se modifica `NodeAbsorptionHandler`; `Node` y las demás ediciones no se ven afectadas.

7.3. Problema de las dependencias excesivas

Antes, `Node.java` importaba 15 clases, incluyendo `DiscretePotentialOperations`, `TablePotential`, `ExactDistrPotential`, `SumPotential`, `UniformPotential`, `PotentialRole` y `LinkRestrictionPotentialOperations`.

Estas dependencias significaban que cualquier cambio en el sistema de potenciales (como la refactorización de `TablePotential` descrita en el documento anterior) podía requerir cambios en `Node`, aunque los cambios fueran irrelevantes para la identidad del nodo.

Ahora `Node` solo importa `Potential` (la interfaz base) y las clases extraídas absorben las dependencias específicas que necesitan.

8. Verificación

8.1. Tests automáticos

Tras cada fase se ejecutaron los 337 tests del módulo `core` y el build completo de los 20 módulos:

Fase	Tests core	Build 20 módulos	Tests integración
Fase 0 (baseline)	337 OK	OK	361 OK
Fase 1 (<code>NodeAbsorptionHandler</code>)	337 OK	OK	—
Fase 2 (<code>VariableTypeConverter</code>)	337 OK	OK	—
Fase 3 (<code>UtilityFunctionComputer</code>)	337 OK	OK	—
Fase 4 (<code>setPotentialConsistently</code>)	337 OK	OK	—
Fase 5 (limpieza)	337 OK	OK	361 OK

8.2. Corrección adicional: `PotentialUtils.getPotentialName()`

Durante la verificación con los tests de integración (361 tests) se detectó que el método `PotentialUtils.getPotentialName()` no recorría la jerarquía de clases al buscar la anotación `@PotentialType`. Las subclases `UncertainTablePotential` y `StrategicTablePotential` (creadas en la refactorización anterior de `TablePotential`) no tenían anotación propia, y el escritor PGMX las grababa con un nombre vacío. Al releer, el lector no encontraba el tipo y lanzaba `PotentialTypeNotSupported`.

La corrección fue hacer que `getPotentialName()` recorra la jerarquía de clases (`getSuperclass()`) hasta encontrar la anotación, de modo que `UncertainTablePotential` hereda el nombre "Table" de `TablePotential`.

9. Resumen de ficheros afectados

Fichero	Acción	Fase
model/network/Node.java	Simplificado	1–5
model/network/NodeAbsorptionHandler.java	Creado	1
model/network/VariableTypeConverter.java	Creado	2
model/network/UtilityFunctionComputer.java	Creado	3
action/core/AbsorbNodeEdit.java	Caller actualizado	1
action/core/VariableTypeEdit.java	Caller actualizado	2
action/core/SetPotentialEdit.java	Caller actualizado	4
potential/operation/LinkRestrictionPotentialOperations.java	Método añadido	4
inference/.../CEPotentialOperation.java	Caller actualizado	5
gui/.../EvidenceManager.java	Caller actualizado	5
gui/.../PotentialEditDialog.java	Caller actualizado	4
potential/plugin/PotentialUtils.java	Bug corregido	5
integrationTests/.../Tools.java	Caller actualizado	5
integrationTests/.../Classifier.java	Caller actualizado	5

Tabla 3: Ficheros afectados por la refactorización de **Node**, organizados por fase.

Referencias

- [1] Robert C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [2] Martin Fowler, *Refactoring: Improving the Design of Existing Code*, 2.^a edición, Addison-Wesley, 2019.
- [3] Arthur J. Riel, *Object-Oriented Design Heuristics*, Addison-Wesley, 1996.